

Problem Statement 2

Controlled Natural Language Generation — Decoder vs LLM for Slot-Conditioned Response Generation

Problem Statement

Build a system that generates natural language responses from structured slot inputs (intent + entities) in a fixed domain. Develop two parallel solutions — a custom lightweight Decoder-based model (small Transformer or seq2seq) and a modern LLM-based approach (prompt-based generation) — and compare them on correctness, fluency, and controllability.

Domain & Dataset Requirements

Fix a single domain (recommended: E-commerce Order Assistant). Define 5–6 intents and 8–10 slot/entity types.

Input format (structured):

```
intent: order_status | slots: {order_id: 1234, status: shipped, date: 2 May}
```

Expected output (natural language):

"Your order 1234 has been shipped on 2 May."

- Dataset size: 200–400 samples, each mapping structured input → natural response
- Variation: Controlled variation across slot combinations; include edge cases (missing/optional slots)
- Split: Standard train/val/test split (e.g., 70/15/15)

Module 1: Data Preparation & Input Representation

Task 1: Dataset Construction & Schema Design (1.5 Marks)

Define the domain intent-slot schema and construct a structured dataset. Each sample must contain a structured input (intent + slot key-value pairs) and one or more reference natural language responses. Include controlled paraphrastic variation and edge cases (missing slots, optional fields, multi-value slots). Provide dataset statistics: intent distribution, slot coverage, and average response length.

Task 2: Input Linearization & Tokenization (1.5 Marks)

Prepare the structured inputs for both modeling paradigms:

- Decoder path: Linearize structured input into a flat token sequence
- LLM path: Design structured prompt templates that encode the slot schema.
- Handle edge cases in linearization: missing slots, empty values, and slot-order sensitivity.

Module 2: Model Development

Task 3: Decoder-Based Response Generator (2 Marks)

Build a compact Decoder-only Transformer (or seq2seq) model that takes linearized structured input and generates a natural language response. Implementation must include:

Task 4: LLM-Based Response Generation (1.5 Marks)

Implement an LLM-based generator using an accessible generative model.

Module 3: Comparative Evaluation & Analysis

Task 5: Quantitative Benchmarking (2 Marks)

Conduct a head-to-head evaluation on the same test set:

- Slot Accuracy (primary): % of responses where all required slots are correctly present with no hallucinated values
- Text Quality: BLEU, ROUGE scores against reference responses. Optionally BERTScore.
- Hallucination Rate: % of responses containing information not present in the input slots
- Efficiency: Inference latency (ms/query), throughput, and relative cost (LLM API vs. local model)
- Test on at least 5–10 edge-case inputs (missing slots, unusual combinations) and report per-intent breakdown.

Task 6: Qualitative Analysis & Deployment Recommendation (1.5 Marks)

Provide a comparative analytical discussion:

- Comparative table: Decoder vs LLM across slot accuracy, fluency, hallucination, latency, and cost
- At least 10 output examples (5 per model) with error diagnosis
- Answer clearly: Which model is more faithful? Which is more natural? Which would you deploy, and why?
- Trade-off analysis: controllability vs. fluency, training effort vs. prompt engineering effort

Note: Students must provide comprehensive explanations and inferences for every task. Your documentation should clearly articulate why Decoder-only architectures are effective for controlled generation tasks and how they compare with LLM-based approaches in terms of faithfulness, controllability, and deployment trade-offs.

Measurement Metrics Summary

Generation Quality

- Slot Accuracy (% correct slot coverage, no hallucination)
- BLEU / ROUGE against reference responses
- Hallucination Rate (% responses with fabricated information)

System-Level Metrics

- Inference latency (ms/query) and throughput
- Model size (parameter count)
- Cost per 1000 queries (API cost for LLM; compute cost for Decoder)

Deliverables for the Assignment

Students must submit:

1. A complete Python notebook (Ipynb Notebook Only)
2. PDF of the ipynb notebook with Results
3. Custom dataset files (with schema documentation)
4. Decoder model architecture implementation from scratch
5. LLM prompting pipeline with prompt templates
6. Training loop with loss calculation (for the Decoder model)
7. **Comparative evaluation results with visualizations**
8. Output examples with analysis
9. A short technical report explaining design decisions, challenges, and limitations

All files must be zipped and uploaded as a single .zip file when submitting the assignment.